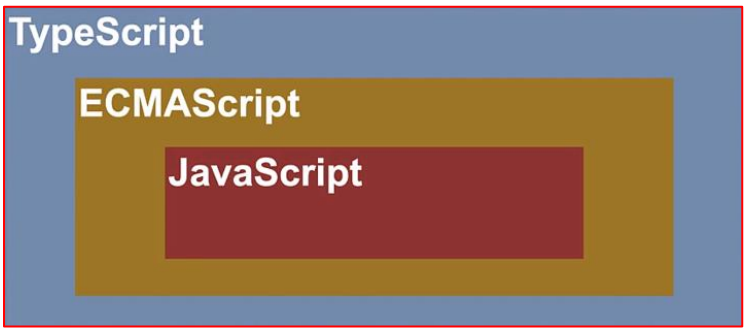
TypeScript Overview

In this lesson, we will cover the basics of TypeScript. TypeScript is a language developed by Microsoft, first released in 2012. It is free and open source. TypeScript provides static typing support to JavaScript, which aids in IDE support for code completion, debugging, and so on. Additionally, TypeScript adds support for object-oriented programming, such as classes, objects, inheritance, interfaces, etc. This makes it easier to use the JavaScript ecosystem or environment in a compiled fashion, which is the real beauty of TypeScript. More details can be found on the official website: `www.typescriptlang.org`

For React development, we can develop using various languages, including JavaScript, an extremely popular programming language. You can also use EcmaScript, the standardized version of JavaScript, referenced as ES6, ES9, etc. Additionally, there's TypeScript, which adds optional types to JavaScript, and you can even develop React apps in other languages such as Dart[1].

Now, TypeScript is the most popular language for React development, and we will discuss why later. But first, let's look at the relationships. TypeScript is a superset of JavaScript and EcmaScript. So, we have the core JavaScript, or plain vanilla JavaScript, then we have EcmaScript, the standardized version of JavaScript, and then we have TypeScript, where we add optional types to the JavaScript programming environment.



You might wonder why most developers use TypeScript instead of Dart, EcmaScript, or plain vanilla JavaScript. The reason is that TypeScript is a strongly typed language with compile-time checking and IDE support. This increases developer productivity and efficiency for very large projects, making TypeScript a good choice for those serious about React development.

Now, let's look at the development process. First, we create our TypeScript code, then we compile the code, and finally, we run the code. This is a high-level, step-by-step process common to any compiled programming language.

[1] Developed by Google, Dart is a client-optimized language for fast apps on any platform. Its main purpose is to offer a more structured but flexible language for the web. Dart can be used to write simple scripts or full-featured apps. It is often used with the Flutter framework for building natively compiled applications for mobile, web, and desktop from a single codebase.

Step One: Creating the TypeScript Code. We create a text file with a `.ts` extension. For example, we have a file here called `myDemo.ts`, where we simply say `'console.log("Hello World.");`

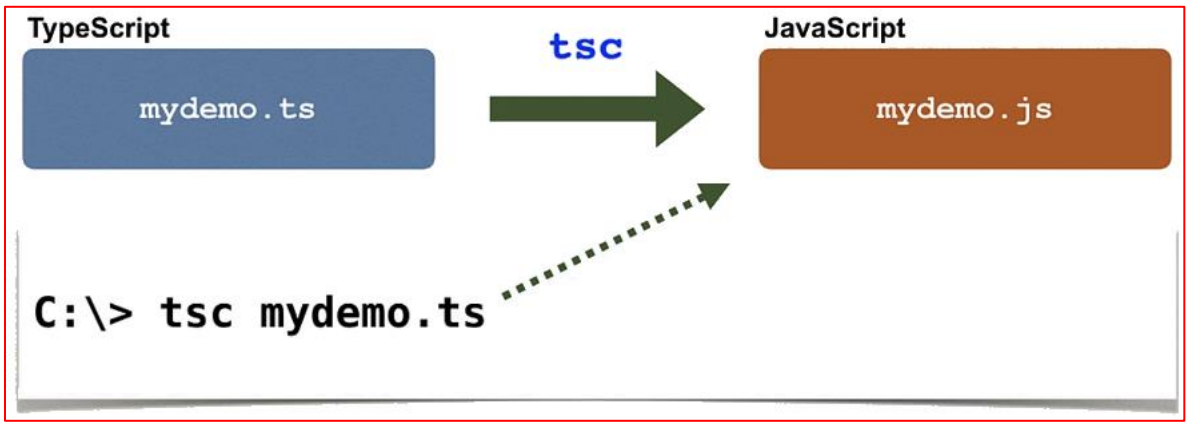
```
console.log("Hello World!");
```

This will log a message to the actual console. It's very simple and straightforward.

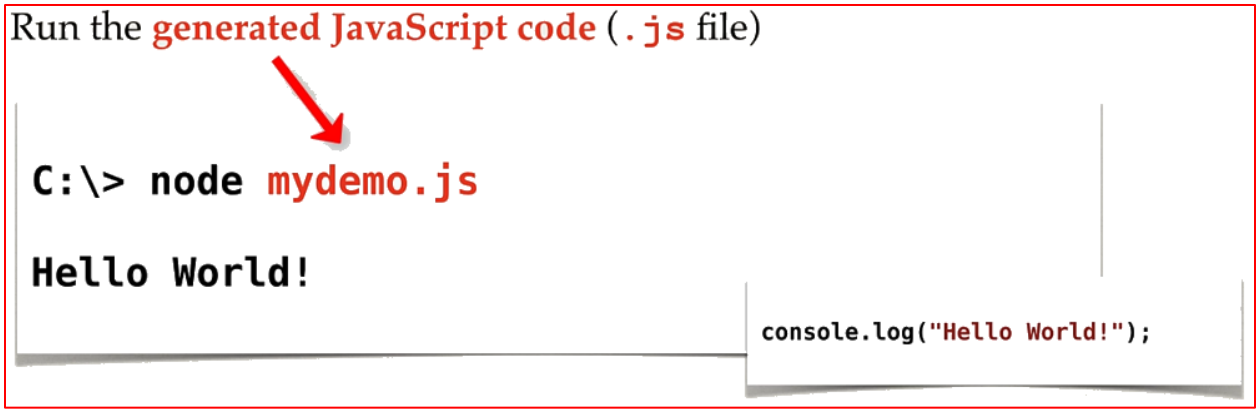
Step Two: Compiling the Code. Web browsers do not understand TypeScript natively; they only understand JavaScript. So, we need to convert our TypeScript code to JavaScript code, a process known as `transpiling`. So, we have our TypeScript code, `myDemo.ts`, and we will convert it to JavaScript code, myDemo.js.



`Transpiling` is accomplished with the `tsc` command. We will use the `tsc` command to `transpile` the code, generating `myDemo.js`, the actual JavaScript code. At the command line, we will simply use `'tsc'` followed by the filename, `'myDemo.ts,'` which will generate the given JavaScript file, `'myDemo.js.'`



Step Three: Running the Code. We will run the generated JavaScript code using the **node** command. At the command line, we will say `'node'` and then reference the JavaScript code, `'myDemo.js,'` which will give us `'Hello World'` because our file had `'console.log Hello World.'`



You might wonder why we even compile. The compiler is your friend; it can find errors at compilation time. For example, if I write some code like `console.logstuff.("Hello World")` and then try to compile it, it will tell me that the property '`logstuff`' does not exist. This is very good because you want to find your errors earlier rather than at runtime, as would happen with regular JavaScript. Again, the compiler is your friend; it helps you find errors early to save you time and headaches.

